Zone: Code–Data Identification, LLM-Assisted Transformation, and Assembly Wrapping for Binary Rewriting

Stefano Bistarelli[0000−0001−7411−9678], Jacopo Di Benedetto, and Francesco Santini[0000−0002−3935−4696]

Dipartimento di Matematica e Informatica, Università degli Studi di Perugia,
Via Vanvitelli 1, 06123 Perugia (PG), Italy
stefano.bistarelli@unipg.it
jacopo.dibenedetto@studenti.unipg.it
francesco.santini@unipg.it

Abstract. Rewriting a raw binary requires distinguishing instructions from embedded data and preserving their references when the layout changes. Zone is a binary-to-binary metamorphic engine for compact Windows x86 and x64 payloads. This extended study adds symbolic/heuristic and LLM-based data identification, LLM-proposed instruction transformations, and integration of assembly wrapping, an existing technique that gives stored bytes different static and executed instruction interpretations. Six annotated benign fixtures are identified exactly in 18 LLM and six symbolic runs. On thirteen separate benign fixtures, LLM transformation yields correct output buffers in 167/168 x64 and 138/144 x86 invocations. The completed campaign on the original 102 payloads exposes a larger reliability gap: wrapping constructs 243/306 variants, with 170 passing the functional harness; LLM transformation constructs 350 outputs across 816 conditions, with 275 passing. Current raw-file scans report no malicious verdicts for 207 distinct wrapping outputs and at least one malicious verdict for 221 of 322 evaluable LLM-transformed outputs; one additional LLM report is empty. These observations separate data identification, construction, observed behavior, and static verdicts. They complement the retained conference evaluation without establishing universal semantic preservation or the behavior of deployed endpoint protection.

Keywords: Binary Rewriting · Metamorphic Code · Code–Data Identification · Large Language Models · Functional Preservation

Draft note. The extension uses the completed campaign logs available on 14 November 2026. The current manifest has 549 completed scan records; 548 contain engine responses and one is empty. Earlier experiments retain their own versions and denominators.

1 Introduction

Code transformations complicate the analysis of malicious software because distinct binary representations may implement the same behavior. Metamorphism is one form

of transformation-based evasion [12]: the program's instruction sequence is rewritten, whereas polymorphic encoding typically changes an encrypted representation and its decoding routine. Both approaches challenge byte-pattern recognition, but they have different implications for the representation available during execution [5,20].

Zone provides a modular environment for examining these effects on compiled Windows x86 and x64 payloads. Its *bin2bin* interface accepts binary input without source code or debugging symbols. The transformations considered here are equal code substitution (EQ), NOP insertion, instruction block permutation (IBP), bogus control flow (BCF), and position-independent instruction (PII). Their individual and combined effects are assessed through detection measurements, functional tests, and changes in size and instruction statistics.

Motivation and Contributions of the Extension. The conference study [4] showed that changes to a payload's representation can be accompanied by loss of functionality. One concrete difficulty is embedded data: a disassembler may read a string as instructions, after which a transformation changes the string instead of code. Even when the bytes are preserved, moving them can invalidate the instructions that refer to them. The present extension makes these preparation and validation problems explicit and evaluates two additional ways to transform the prepared code.

The contributions are:

- A data-preparation layer with alternative symbolic/heuristic and LLM classifiers. It identifies candidate data regions and uses shared deterministic handling to preserve supported data and references.
- An LLM transformation path for EQ, NOP, BCF, and IBP, comparing requests without examples and with two worked examples. Generated assembly is checked before a separate functional assessment.
- Integration of assembly wrapping for x64 and x86. This existing technique changes instruction boundaries so that a static listing and execution can interpret the same stored bytes differently.
- A completed evaluation on annotated benign fixtures and the original 102-payload corpus, reporting classification, build availability, functional outcomes, static scans, output diversity, and recorded cost.

The article retains the original architecture, PII design, detailed conference results, and comparison with Shikata Ga Nai as its foundation. Reused figures, tables, listings, and the algorithm carry source citations. The added methods and experiments are identified separately, making clear which claims come from the conference study and which are supported by the completed extension.

The scope is restricted to compact, position-independent payloads rather than complete Portable Executable (PE) files. Full-file rewriting requires additional treatment of section boundaries, imports, relocation information, alignment, and entry points. Those format-dependent issues would complicate the interpretation of experiments intended to compare instruction-level transformations. Signatures in non-code regions of PE files are an additional consideration [22].

Operating on largely executable instruction sequences makes it possible to examine a substantial fraction of each input. It also limits the conclusions: these experiments

address a payload-level representation, not the complete delivery and execution of a malicious application. In particular, functional preservation and the detectability of a raw binary must be considered separately.

The published baseline comprises 102 Windows payloads, including 38 x64 and 64 x86 inputs, evaluated under fifteen transformation configurations. Its main observation is a trade-off between changes in the binary representation and the proportion of variants that remain functional. Some configurations received no malicious verdicts in the VirusTotal scans, but such observations are specific to the file format and testing conditions; they do not establish general resistance to deployed endpoint products.

Sections 2 and 3 retain the background and engine design. The original experiments and representation measurements appear in Sections 4 and 5, followed by their limitations and the SGN comparison (Sections 6 and 7). Sections 8, 9, and 10 describe the additional methods. Sections 11 and 12 describe its design and completed results. Section 13 interprets their limitations, Section 14 discusses related work, and Section 15 concludes.

2 Background

The following background distinguishes metamorphic rewriting from polymorphic encoding and introduces the transformation families used in the baseline evaluation.

2.1 Metamorphic Engines

A metamorphic engine constructs a different instruction-level representation of an input program intended to preserve its behavior. Early virus generators used this idea to vary their byte sequences between instances [25]. Functional preservation is a design objective and must be checked; it does not follow merely from the fact that a transformation has produced an executable output.

A key difference is the absence of a mandatory decrypt-and-execute stage [29]. An encrypted payload is normally recovered before execution, while a successfully rewritten payload can execute in its transformed representation. This changes what a memory scan may observe, without implying that the transformed program is undetectable.

Instruction substitution, insertion, and reordering can generate many representations from the same input [29,17]. This diversity complicates the use of fixed byte patterns, and places a substantial burden on the rewriting engine to preserve control flow, data references, and processor state. Transformation-based evasion and its limitations are surveyed in [12].

Comparison with Polymorphism. Polymorphic encoding usually combines an encoded body with a variable decoding routine [25]. Shikata Ga Nai is a representative example: variation in its decoding stub changes the static byte sequence, but recurring features of that stub can themselves be used for recognition [18]. Once decoding has completed, the original body may be available to memory analysis. Metamorphic rewriting instead changes the body that is executed [11]; this is a distinction between representations, not a guarantee of runtime evasion.

Table 1. Instruction substitutions and corresponding encodings (x86). Reproduced from Table 1 of [4].

Original Inst.	Morphed Inst.	Orig. Bytes	Morphed Bytes
`XOR eax, eax`	`MOV eax, 0`	`31 C0`	`B8 00 00 00 00`
`SUB eax, 1`	`ADD eax, -1`	`83 E8 01`	`83 C0 FF`
`ADD eax, 2`	`SUB eax, -2`	`83 C0 02`	`83 E8 FE`
`MOV eax, 0`	`PUSH 0; POP eax`	`C7 C0 00..`	`6A 00 58`

2.2 Morphing Techniques

The established transformation families considered in Zone are EQ, NOP insertion, IBP, and BCF. They can be evaluated separately or in selected combinations. Their antecedents are discussed in Section 14.

EQ changes the instruction vocabulary; NOP insertion adds operations intended to leave the relevant computation unchanged. IBP modifies the physical layout of instruction blocks, whereas BCF introduces branches and unreachable regions. PII, described in Section 3.1, further separates instruction placement from logical execution order. These descriptions summarize the design reported in [4].

Equal Code Substitution. EQ replaces an operation with an alternative representation intended to have the same effect. For example, the zeroing operations `XOR rax, rax` and `MOV rax, 0x0` have different encodings. Such examples illustrate changes in byte representation; preservation of all relevant state remains an implementation concern.

Table 1 collects the substitution examples used in the conference study, including arithmetic complements and stack-based sequences. Instruction substitution and its use with insertion transformations have a long history [7,8,23].

NOP/Dead Code Insertion. Insertion transformations introduce operations intended to leave the program's relevant behavior unchanged. The single-byte `NOP` is one example; Table 2 reproduces the other examples considered in the original implementation.

Some apparent no-ops, including arithmetic with zero, can change processor flags. The original implementation therefore used flag-preserving sequences around such operations, as illustrated in Table 3. The resulting repeated patterns are themselves relevant to detection: changes introduced by an engine can create recognizable regularities [3].

Instruction Block Permutation. IBP changes where instruction blocks are stored while retaining their intended logical succession. The original implementation organizes the transformation into the following stages:

1. **Partitioning.** The instruction sequence is divided into blocks; the chosen granularity affects both rewriting overhead and the amount of layout variation.
2. **Connecting blocks.** An unconditional branch links each block to its logical successor.
3. **Changing placement.** Blocks are permuted, while the entry block retains its position.

Table 2. NOP-like operations and their encodings. Reproduced from Table 2 of [4].

Example	Compiled Opcode
NOP	0x90
SUB rax, 0x0	0x48, 0x83, 0xE8, 0x00
ADD rax, 0x0	0x48, 0x83, 0xC0, 0x00
MOV rax, rax	0x48, 0x89, 0xC0
LEA rax, [rax + 0]	0x48, 0x8D, 0x40, 0x00
PUSH rax; POP rax	0x50, 0x58
OR rax, rax	0x48, 0x09, 0xC0
XOR rax, 0x0	0x48, 0x83, 0xF0, 0x00
AND rax, rax	0x48, 0x21, 0xC0
INC rax; DEC rax	0x48, 0xFF, 0xC0; 0x48, 0xFF, 0xC8
DEC rax; INC rax	0x48, 0xFF, 0xC8; 0x48, 0xFF, 0xC0
SAR rax, 0x0	0x48, 0xC1, 0xF8, 0x00
SHR rax, 0x0	0x48, 0xC1, 0xE8, 0x00
SHL rax, 0x0	0x48, 0xC1, 0xE0, 0x00
RCL rax, 0x0	0x48, 0xC1, 0xD0, 0x00
RCR rax, 0x0	0x48, 0xC1, 0xD8, 0x00

Table 3. Flag-preserving insertion example. Reproduced from Table 3 of [4].

Instruction	Compiled Opcode
PUSHF	0x9C
ADD rax, 0x0	0x48, 0x83, 0xC0, 0x00
POPF	0x9D

4. **Updating references.** Control-flow targets are adjusted to the resulting layout.

This transformation changes adjacency without necessarily changing most instructions within a block. Consequently, a detector based on a short pattern internal to an unchanged region may behave differently from one that relies on a longer contiguous sequence. The experiment assesses these changes empirically rather than assuming that every signature is disrupted.

Bogus Control Flow. BCF adds branches around code that is not intended to execute. The implementation uses conditions with predetermined outcomes to exclude the inserted region from the execution path [10,15]. Table 4 reproduces the illustrative block from the conference article: the condition transfers control past the padding at addresses 0x11–0x17.

1. **Inserted regions.** The engine creates the additional blocks, which may contain instruction sequences or raw bytes.
2. **Predetermined conditions.** Conditions are constructed to select a known branch outcome.
3. **Integration.** The conditions and corresponding branches are added to the instruction stream. Table 4 illustrates the resulting organization.

Table 4. Illustrative BCF block. Reproduced from Table 4 of [4].

Address	Instruction
0x0	`mov rax, 0x1`
0x7	`cmp rax, 0x1`
0xb	`je 0x18`
0x11	`nop`
0x12	`nop`
0x13	`nop`
0x14	`nop`
0x15	`nop`
0x16	`nop`
0x17	`nop`
0x18	`original code.......`

3 Zone Architecture

The name Zone refers to Tarkovsky's *Stalker* and its alternative paths toward a common destination. The open-source implementation[1] was originally explored for both complete PE files and raw payloads. The extended manuscript retains the payload-level scope of the conference paper [4]; the difficulties of maintaining PE metadata are not evaluated as a separate contribution.

The implementation is written in Python. Capstone disassembles the input, the selected transformations modify the instruction representation, and Keystone assembles the output. The *bin2bin* interface requires neither source code nor Program Database (PDB) symbol files.

3.1 Position-Independent Instruction

PII was introduced in the conference paper [4]. It decouples the physical position of an instruction from its place in the intended execution sequence.

Organization. Consider an instruction such as `ADD rax, 1`. In the PII representation it is placed in a wrapper rather than relying on adjacency to its successor. The wrapper executes the original operation and transfers control to a redirector. An offset table supplies the location of the next logical instruction. Figure 1 reproduces the overall layout, while the listings below document the components of the original design.

The representation consists of four components:

1. **Base-Address Stub (Stub1).** This component determines the runtime base address and records it in a designated memory location.
2. **Redirector Stub (Stub2).** This component obtains the next logical instruction location from the offset table and dispatches execution to it.
3. **Offset Table.** This contiguous table contains the instruction offsets used by the redirector.

[1] Project repository: https://github.com/jakydibe/Zone.

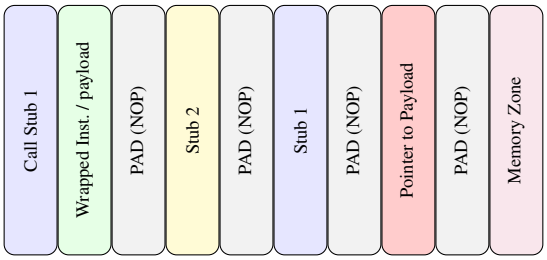


Fig. 1. PII payload layout. Adapted from Fig. 1 of [4].

4. **Wrapped Instructions.** Each input instruction is placed in a wrapper intended to maintain the required register, flag, and stack state.

Execution Sequence. The conference paper describes execution in three phases:

- **Initialization.** A call enters Stub1 (Listing 1). The return address supplied by that call is used to establish the payload's runtime location.
- **Original operation.** The wrapper in Listing 2 restores the saved state, adjusts the stack, executes the input instruction, and supplies its logical index to Stub2.
- **Dispatch.** The redirector in Listing 3 saves the state it uses, obtains the successor's location from the offset table and base address, and transfers control to that wrapper after restoring the relevant state.

Physical Placement. The original implementation permutes complete wrappers, treating each five-instruction wrapper as a unit. The offset table is updated for the resulting placement. Execution still starts with the initial call to Stub1; logical succession is represented by the table rather than by the wrappers' physical adjacency.

Listing 1. Base-address component. Reproduced from Listing 1 of [4].

```
pop <random_unused_reg>;    popping 2nd instruction address
mov [<random_unused_reg> - 5 + <init_address>],  <random_unused_reg>; move it to a memory
     slot
push <random_unused_reg>; push just to avoid altering the stack
jmp <random_unused_reg>; return to the caller (2nd instruction)
```

Listing 2. Instruction wrapper. Reproduced from Listing 2 of [4].

```
pop rax;                    ; popping rax because we just used it to save jmp address
lea rsp, [rsp + 8];         ; adjusting stack to avoid stack overflow
ORIGINAL_INSTRUCTION;       ; original instruction
push <0x3e8 + counter>;     ; pushing instruction original index + a constant
jmp <stub2_address>;        ; Jumping to 'redirector' stub
```

Listing 3. x64 redirector. Reproduced from Listing 3 of [4].

```
push rax;                                   saving original 'rax' register state
push rcx;                                   saving original 'rcx' register state
mov rcx, [rsp + 0x10] ;                     getting the instruction index from the stack.
pushf;                                      saving original CPU flags
sub rcx, 0x3e8; ;                           subtracting a constant to get instr index
```

```
inc rcx;                                        increase instruction number
mov rax, [rip + init_addr_offset];              get payload base address
mov rcx, [rax + mem_address - 5 + rcx * 8];     get next instruction address
sub rcx, 0x5;                                   just because base address is 2nd instruction
add rax, rcx;                                   add payload base address to the offset
popf;                                           restore cpu flags
pop rcx;                                        restore rcx value
jmp rax;                                        jmp to next instruction
```

Architectural Variants. Listings 3 and 4 show the published x64 and x86 redirectors. In addition to the different register widths, they differ in how the required address is obtained: the x86 version uses the call-based mechanism documented in the original listing rather than x64 RIP-relative addressing.

Listing 4. x86 redirector. Reproduced from Listing 4 of [4].

```
push eax;
push ecx;
mov ecx, [esp + 0x8]
pushf;
sub ecx, 0x3e8;
inc ecx;

call <stub2_address + 0x14>;    calling pop eax, 0x14 is pop eax offset from stub2 address
pop eax;                        popping eax puts in eax the value of the instruction pointer

mov eax, [eax + <init_addr_offset>];
mov ecx, [eax + <mem_address> - 5 + ecx * 4];
sub ecx, 0x5
add eax, ecx
popf;
pop ecx;
jmp eax;
```

3.2 How Instructions Are Patched

Moving instructions or inserting bytes can invalidate references to their previous locations. The patching phase updates control-flow destinations and other affected references to the modified layout. Branches and calls are therefore part of the rewriting process, not merely instructions that can be copied without regard to their new positions.

This requirement also explains the scope restriction. Rewriting raw instruction sequences already requires consistent target updates; complete PE files introduce additional metadata and layout dependencies. Algorithm 1 reproduces the high-level workflow from [4].

3.3 Transformation Pipeline

The original workflow in Algorithm 1 can be read as six stages. The additional analysis and transformation paths are described in Sections 8–10.

1. **Input representation.** Capstone produces instruction objects held in a linked sequence, with an address-indexed dictionary for lookup (Line 4).

Algorithm 1: High-level Zone workflow. Reproduced from Algorithm 1 of [4].

```
   Data: raw_bytes
   Result: Transformed binary
1  global instruction_list ← [];
2  global instruction_dict ← {};
3  global label_table ← [];
4  Function disass
5      CAPSTONE.SKIPDATA ← TRUE;
6      for each instruction in disasm(raw_bytes)
7          instruction_list.append(instruction);
8          instruction_dict[instruction.address] ← instruction;
9          if instruction is X86.Control_flow_instruction
10             jump_address ← extract_address(instruction.op_string);
11             label_table.append((instruction, instruction_dict[jump_address]));
12 Function update_instructions
13     new_instruction_dict ← {};
14     for each instruction in instruction_list
15         address ← instruction.previous_instruction.address + len(instruction.previous_instruction.bytes);
16         instruction.address ← address;
17         new_instruction_dict[instruction.address] ← instruction;
18     instruction_dict ← new_instruction_dict;
19 Function morph
20     equal_code_substitution();
21     nop_insertion();
22     instruction_block_shuffling();
23     bogus_control_flow();
24     position_independent_instruction();
25 Function adjust_label_table
26     update_instructions();
27     for each entry in label_table
28         jmp_addr ← extract_address(entry[0].op_string);
29         jmp_addr_new ← entry[1].address;
30         new_instruction ← keystone.assemble( “entry[0].mnemonic jmp_addr_new”);
31         entry[0] ← new_instruction;
32 Function main
33     disass();
34     morph();
35     adjust_label_table();
36     write_to_file();
```

2. **Embedded data.** The implementation enables Capstone's `Skipdata` option (Line 5). This does not solve the general problem of identifying inline data, particularly when data bytes also admit an instruction interpretation [10].
3. **Reference tracking.** A label table associates each tracked control-flow operation with its target instruction object. This association is retained as the object's physical address changes.
4. **Transformation and layout update.** The chosen transformations are applied (Line 19), followed by recomputation of instruction addresses (Line 12).
5. **Reassembly of tracked branches.** Keystone assembles the tracked control-flow operations using their updated targets (Line 25).
6. **Output.** The resulting instruction bytes are written as a new raw binary (Line 36).

4 Published Experimental Baseline

Draft note. The detailed historical plots and their reported numerical values are retained from the preparation manuscript. Some category/value associations in the detection and working-rate plots, and the provenance of the similarity metric, require reconciliation against the conference version and archived records. These are baseline editorial checks, not replacement experiments. See `BASELINE_CHECKS.md`.

This section restates the protocol and results of the conference paper [4]. It introduces no additional measurements. VirusTotal scans supplied verdicts from more than 70 engines, subject to the file-format limitations discussed below.

The corpus contains 102 Windows payloads: 38 x64 inputs and 64 x86 inputs. Most originate from the Metasploit Framework and include reverse and bind shells. This provides an identifiable collection of payload types, but does not constitute a representative sample of all Windows malware.

Two scope restrictions are relevant to interpretation:

- **Architecture coverage.** The x64 and x86 sets differ in size and membership. Architecture-level comparisons therefore also reflect differences between the two input collections.
- **Excluded inputs.** Large, DLL-based command-and-control components are outside the payload-level scope. Their PE structure and embedded data introduce additional rewriting dependencies.

Testing Method. Each of the fifteen configurations was applied to every input, with ten generated variants per input and configuration to expose variation in the transformation process. The five individual transformations and ten combinations were:

- **Single:** NOP (`nop`), Equal Code Substitution (`eq`), Bogus Control Flow (`bcf`), Instruction Block Permutation (`ibp`), and Position-Independent Instruction (`pii`).
- **Combined:** `eq_nop_bcf`, `eq_nop_ibp`, `eq_nop_pii`, `eq_nop`, `eq_bcf`, `eq_ibp`, `eq_pii`, `nop_bcf`, `nop_ibp` and `nop_pii`.

The *detection rate* for an individual file is the fraction of scanning engines that returned a malicious verdict; summaries aggregate these per-file measurements. Figure 2 presents the published transformed-file results. The corresponding baseline averages for the original files were 21.67% for x64 and 22.63% for x86.

The plotted observations vary by transformation and architecture. Several configurations involving PII received no malicious verdicts in these raw-file scans, whereas EQ retained more detections. These measurements describe the tested representations and submission conditions; they should be interpreted together with the functional results.

The *working rate* is the proportion of variants that passed the original execution checks (Figure 3). Those checks observed the expected connection for reverse shells, port opening for bind shells, and exception-free termination for the remaining generic cases. The tests are empirical functional checks rather than proofs of semantic equivalence. The conference paper reported greater overall stability for EQ and lower stability for PII, particularly in the x86 collection; the results also varied substantially across combined configurations.

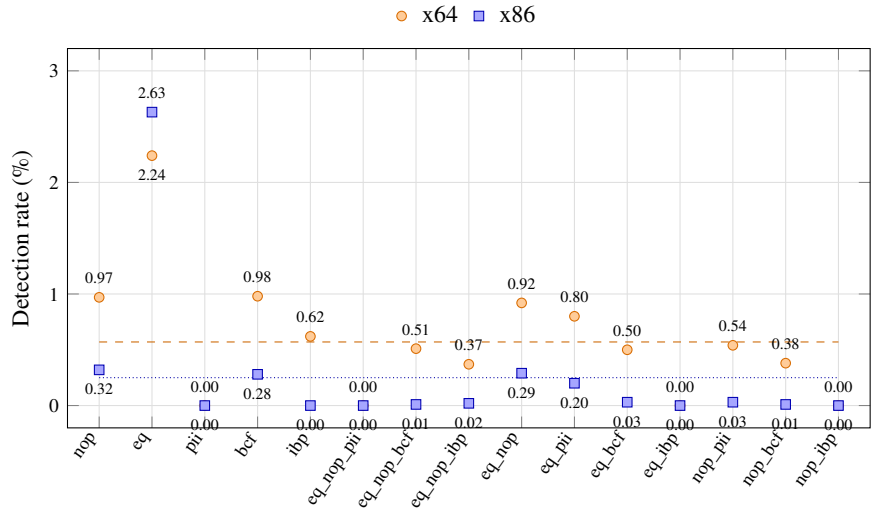


Fig. 2. Malicious-verdict fractions in raw-file scans. Dashed reference lines show the reported architecture averages (x64: 0.57%; x86: 0.25%). Data from Fig. 2 of [4]; redrawn for the present format.

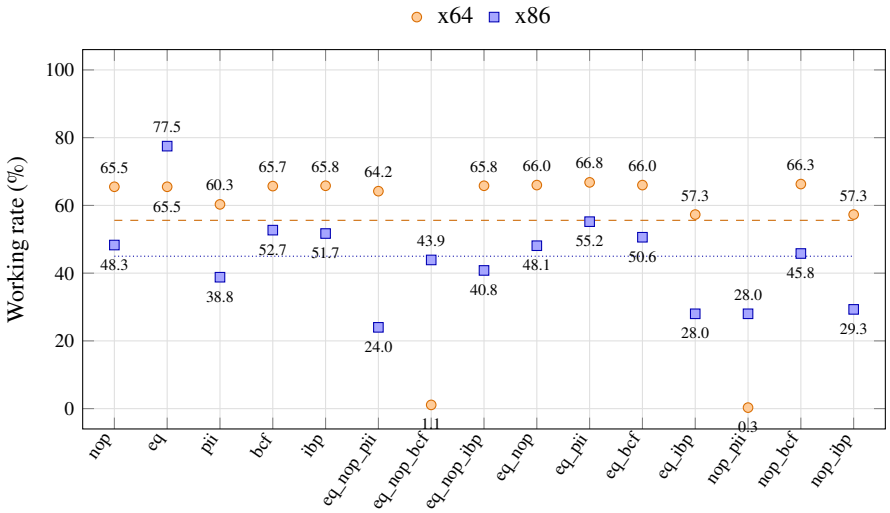


Fig. 3. Variants passing the published functional checks. Dashed reference lines show the reported architecture averages (x64: 55.6%; x86: 45.0%). Data from Fig. 3 of [4]; redrawn for the present format.

Evaluation Limitations. A raw `.bin` submission is not a complete executable application. Engines may apply a different analysis path to such input, omit execution-based analysis, or decline to analyze an unsupported representation. Low detection fractions therefore cannot be interpreted as an evaluation of complete antivirus or endpoint detection and response (EDR) products during deployment.

Submitting original and transformed payloads under the same format assumptions provides a limited basis for comparing the representations in this corpus. It does not remove format-dependent biases or establish behavior under other execution conditions. The published results should be read within those boundaries [4].

5 Size and Statistical Measurements

The conference paper also characterized how the transformed representations differ from their inputs [4]. Its three additional measurements were:

- Payload length variation.
- Delta entropy.
- Cosine similarity.

Payload Length Variation. Let L_i denote the byte length of a transformed file and let n be the number of files considered for a configuration. The reported mean length is:

$$\overline{L} = \frac{1}{n} \sum_{i=1}^{n} L_i$$

The size results in Figure 4 show substantial growth for PII and BCF configurations. Some EQ-containing configurations instead produced smaller average outputs than the original-file reference. The conference paper associated this observation with the implementation's treatment of data during disassembly; the size measurement alone does not establish that any removed bytes were semantically irrelevant.

Delta Entropy. The reported entropy change compares the transformed representation, H_{tech}, with its original, H_{orig}, using the percentage difference:

$$\Delta H\ (\%) = \frac{H_{tech} - H_{orig}}{H_{orig}} \times 100 \tag{1}$$

The measurements were computed on raw bytes and on tokenized assembly. Figures 5 and 6 retain the published results. The assembly-based representation also permits comparison with the metrics discussed by Mohseni et al. [19].

In the assembly results, NOP insertion increased entropy for both architectures, while PII decreased it. The latter observation is consistent with the repeated bookkeeping instructions in the wrapper representation. A small entropy change does not, by itself, imply a small change in control-flow organization. The conference paper illustrated this limitation using BCF and IBP:

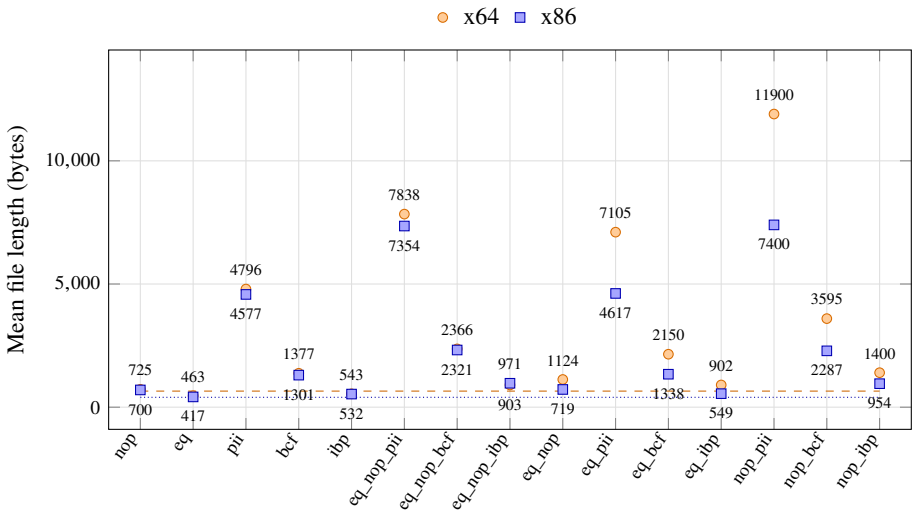


Fig. 4. Mean transformed-file length. Reference lines indicate the original-file means (x64: 652.632 bytes; x86: 400.750 bytes). Data from Fig. 4 of [4]; redrawn for the present format.

- **BCF.** A small entropy difference coexisted with greater length and a changed opcode distribution (Figure 7).
- **IBP.** Moving blocks can change their layout while retaining much of their instruction-frequency distribution.

Entropy is consequently treated as a descriptive statistic of the chosen representation, not a standalone measure of structural complexity or functional preservation.

Cosine Similarity. The frequency vectors A and B represent the opcode distributions of an original file and its transformed variant. Their cosine similarity is:

$$\mathrm{sim}(A, B) = \frac{A \cdot B}{\|A\| \, \|B\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2} \sqrt{\sum_{i=1}^{n} B_i^2}}$$

Here A_i and B_i are the frequencies of opcode i, and n is the size of their joint vocabulary. The score summarizes agreement between the two frequency profiles. Its use in assembly-obfuscation evaluation is discussed in [19]; it does not establish that the programs have the same behavior.

Instruction-frequency agreement is insufficient to validate execution: a small error can change a program's behavior while leaving that distribution almost unchanged. For this reason, the baseline used the execution checks described in Section 4 to assess functionality separately.

Figure 7 shows higher similarity for EQ and IBP than for the insertion-heavy configurations. This is consistent with their different effects on the instruction vocabulary. The

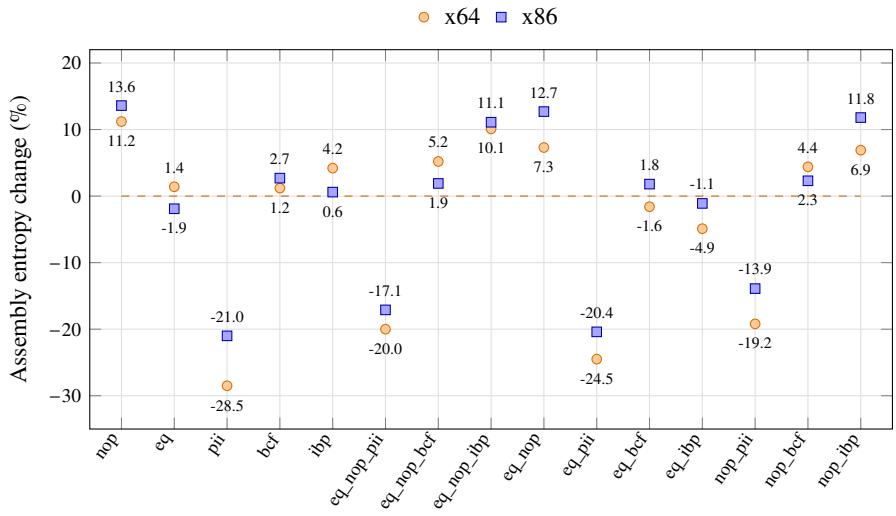


Fig. 5. Percentage entropy change for tokenized assembly; the reference level is zero. Data from Fig. 5 of [4]; redrawn for the present format.

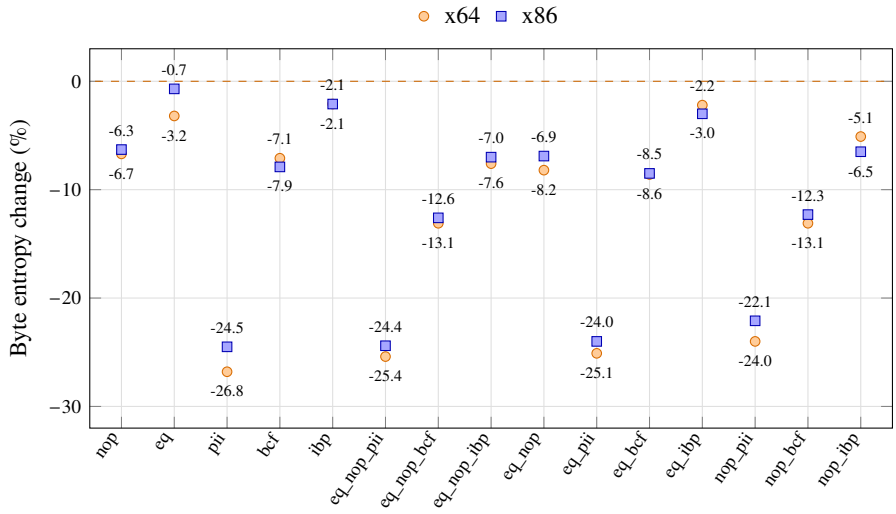


Fig. 6. Percentage entropy change for raw bytes; the reference level is zero. Data from Fig. 6 of [4]; redrawn for the present format.

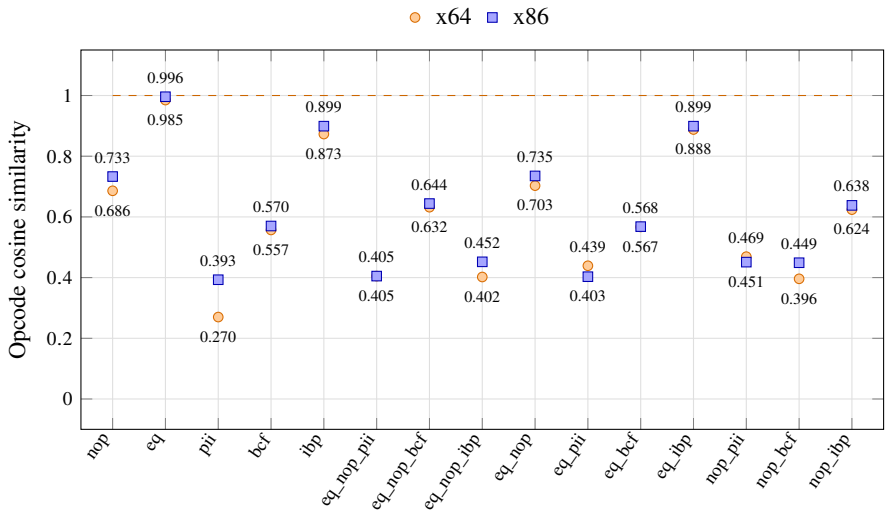


Fig. 7. Cosine similarity between original and transformed opcode-frequency vectors; the reference level is one. Data from Fig. 7 of [4]; redrawn for the present format.

larger size changes in Figure 4 provide complementary information about the additional instructions; neither measurement should be substituted for a functional test.

6 Limitations and Defensive Implications

The following limitations concern the original implementation and experiments [4]. Sections 8–10 describe subsequent work motivated by these observations; the residual limits of the added paths are discussed in Section 13.

- **Instruction coverage.** Less common instructions, including the example `JRCXZ`, can require handling not exercised by this corpus. Testing a fixed collection of 102 inputs does not establish complete x86/x64 coverage.
- **Code–data ambiguity.** Embedded strings may partly decode as instructions. Table 5 illustrates an input in which the bytes of `.exe` are interpreted as a branch. Rewriting the resulting instruction interpretation can corrupt the embedded string.

Accurate recovery of instructions and references is a broader difficulty in static binary analysis [28]. General limitations of disassembly do not establish that perfect results are impossible on a bounded test corpus; they also do not excuse uncharacterized implementation failures. The reported working rates describe this implementation and these inputs.

The two examples below document the observed code–data distinction. They illustrate one failure mechanism, not a general guarantee about other payloads.

In Table 5, `calc.exe` is stored at the end of the input. Although some characters are represented as data, part of the string receives an instruction interpretation. This explains why the original implementation could modify bytes that were meant to remain data.

Table 5. Inline-data example affected by rewriting. Reproduced from Table 5 of [4].

Bytes	Instruction
`0x109 b'\xff\xd5'`	call rbp
`0x10b b'c'`	.byte 0x63
`0x10c b'a'`	.byte 0x61
`0x10d b'l'`	insb byte ptr [rdi], dx
`0x10e b'c'`	.byte 0x63
`0x10f b'.exe'`	js 0x178
`0x113 b'\x00'`	.byte 0x00

Table 6. Immediate-operand example. Reproduced from Table 6 of [4].

Bytes	Instruction
`0x8d b'Q'`	push rcx
`0x8e b'H\xb9calc.exe'`	movabs rcx, 0x6578652e636c6163
`0x98 b'Q'`	push rcx
`0x99 b'H\x8d\x0c\$'`	lea rcx, [rsp]
`0x9d b'H1\xd2'`	xor rdx, rdx
`0xa0 b'H\xff\xc2'`	inc rdx
`0xa3 b'H\x83\xec('`	sub rsp, 0x28
`0xa7 b'\xff\xd0'`	call rax

Table 6 shows the other published example, in which the string occurs as an immediate operand at address `0x8e`. The reported transformation did not encounter the same inline-data failure. This observation is specific to that example and does not imply general immunity from rewriting errors.

6.1 Mitigations and Countermeasures

Changing the binary representation does not remove a program's observable actions. Defensive analysis can consider memory protections, execution patterns, API activity, and network behavior in addition to static bytes. Recognition of a transformation engine is another possibility: repeated artifacts of a common toolchain can persist across outputs, as documented for Shikata Ga Nai [18]. Statistical or learned detectors must also account for benign programs that share similar features. These are defensive considerations arising from the study, not results of an additional detection experiment.

7 Comparison with Shikata Ga Nai

The conference paper [4] compared Zone with the Shikata Ga Nai (SGN) polymorphic encoder [13]. SGN combines additive-feedback encoding with a variable decoding stub. The comparison included the original x86 encoder and an unofficial x64 implementation [9]; recognition of the x86 stub has been discussed in [18].

The published aggregate static-scan observations were:

- **x64:** SGN achieved 0%, while Zone averaged 0.57% (reaching 0% with the PII technique).
- **x86:** SGN had a 10.22% detection rate, while Zone achieved 0.25% (also reaching 0% with PII).

The encoders also differ in the representation available after decoding. The conference paper submitted memory dumps obtained during SGN execution to VirusTotal and reported verdicts matching those for the unencoded inputs [4]. This observation is consistent with exposure of the original representation after decoding and with prior work on extracting attack code from memory [11].

Zone's intended execution model retains the rewritten representation. The dump experiment nevertheless measures the recognizability of submitted memory contents; it is not a direct measurement of an endpoint product's runtime detection. Claims about deployed EDR behavior require evidence beyond this comparison.

8 Code–Data Identification and Data-Preserving Rewriting

8.1 Why Data Identification Comes Before Transformation

A raw binary does not tell a rewriting tool which bytes are instructions and which are data. A short program may contain both executable code and an embedded string or lookup table. A disassembler translates bytes into assembly instructions, but the ability to decode a byte sequence does not establish that the processor will execute it. String bytes may also decode as plausible instructions. Rewriting those apparent instructions can therefore corrupt a value that the program subsequently reads.

This is the failure illustrated by the inline string in Table 5. It differs from Table 6, where the string is encoded inside an instruction operand: there, the surrounding instruction must remain intact. A useful identification method must distinguish these situations, rather than marking every printable sequence as a separate data object.

The extension introduces a preparation stage with two responsibilities. *Identification* proposes where data occurs. *Data handling* decides which proposed objects the engine can retain through a layout change, preserves their contents, and updates supported references to their new locations. A reference is an instruction operand or address calculation that points to a code or data location. Preserving a string's bytes is insufficient if the program subsequently reads its old address.

Figure 8 locates these responsibilities in the extended engine. The symbolic/heuristic and LLM classifiers are alternative ways to propose data regions. Separately, the transformation itself can be implemented by the native engine or proposed by an LLM. Thus, using an LLM to identify data does not imply using it to rewrite instructions.

8.2 What a Classification Result Means

For an input of n bytes, a proposed data region is an interval $[s, e)$: it starts at offset s from the beginning of the input and includes every byte up to, but not including, e. For example, $[2, 8)$ denotes six bytes. An optional type describes a string encoding, such as

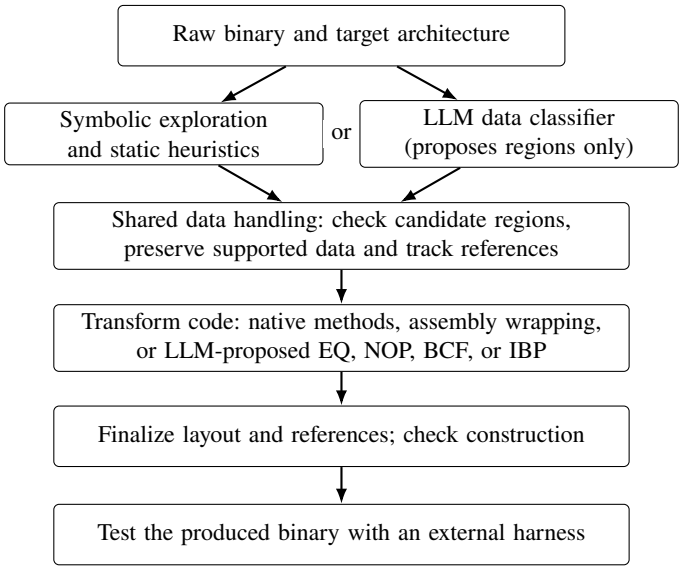


Fig. 8. Conceptual organization of the extensions. Identification chooses candidate data regions; shared handling preserves supported objects during rewriting. Successful construction is followed by a separate behavioral test. Unsupported inputs can stop before a binary is produced.

ASCII or UTF-16LE, or another data object. The output is a set of candidate intervals, not a replacement binary.

Three stages must be distinguished: a classifier's proposed regions, regions accepted by the engine's checks, and regions actually extracted from the instruction stream. A correct proposal may still be unsupported if its references cannot be updated. Conversely, different proposals can produce the same binary when the shared checks retain the same objects. The experiments compare annotations, accepted outputs, and final hashes at their respective stages.

8.3 Symbolic Exploration Combined with Static Heuristics

The non-LLM method combines three sources of evidence. First, static analysis recovers a control-flow graph (CFG): nodes represent basic blocks of instructions and edges represent possible transfers of execution. Zone uses angr's CFGFast analysis for this initial structure [2]. Second, symbolic execution explores possible paths while representing unknown register and memory values as symbolic expressions. This supplies additional evidence about which instructions can be reached. Third, pattern-based rules look for candidate strings and graph-inferred data.

An observed instruction protects its entire decoded byte span, not only its starting address. This matters for variable-length instructions: an interior byte should not become a data candidate merely because the explorer recorded only the instruction's first byte. Newly observed entry points can also support another graph-recovery pass. Printable

candidates that overlap observed code are excluded by the string heuristic; candidates overlapping graph-recovered code need additional structural support.

Exploration is bounded by time, solver, and state limits. The initial state does not reproduce every feature of a Windows process, including all external API behavior. Consequently, an unvisited region is *unknown*, not proven to be data. Reports retain the evidence category, stopping reason, unresolved states, and errors. These distinctions prevent a partial execution search from being presented as a complete partition of the binary into code and data.

8.4 LLM Identification and Shared Data Handling

The alternative classifier receives the raw bytes in hexadecimal, a linear disassembly, and candidate printable strings. Linear disassembly means decoding instructions in byte order; its listing is evidence for the model to assess, not a trusted annotation. The requested answer contains structured data intervals. The model does not perform extraction or rewrite the program in this mode.

Deterministic parsing rejects intervals outside the input, empty intervals, and invalid overlaps. The engine then examines supported references and applies its data-handling rules to a fresh working instance. Accepted data contents are retained separately from the instruction representation; supported references are finalized after transformation, when the new locations are known. Integrity checks verify the preserved contents. Architecture-specific addressing requires separate x64 and x86 handling.

For a benign example, consider a routine that copies the embedded strings `alpha` and `beta` into an output buffer. Identification finds the two string regions; handling preserves both strings and their links to the copying code; transformation changes the instruction representation. The final test checks that the intended strings reach the intended buffer positions. Correct interval identification, successful reference handling, and correct observed output are three distinct questions in this example.

The general classifier can receive validation feedback and make another proposal. Its worked-example setting is independent of the zero-shot and two-shot transformation conditions defined next. Cached identification can reuse regions associated with an input identity, while references are recomputed for the working representation. These shared steps also explain why agreement between final binaries is not, by itself, a measure of the LLM classifier's accuracy.

9 LLM-Assisted Assembly Transformation

9.1 What the Model Is Asked to Change

In transformation mode, the LLM receives an assembly representation and proposes a new instruction sequence intended to preserve the program's behavior. This differs from Section 8, where its answer contains only data locations. Zone supports four transformation requests: equal code substitution (EQ) replaces instruction sequences; NOP insertion adds operations intended to leave the relevant state unchanged; bogus control flow (BCF) adds control-flow structure without changing the intended outcome; and instruction block permutation (IBP) changes the placement of blocks while retaining their

execution relationships. These are the families introduced in Section 2; the LLM proposes their application rather than introducing a fifth family. PII and assembly wrapping remain native engine operations.

Before the request, Zone prepares a *canonical assembly listing*: a working representation with symbolic labels for relevant locations and the extracted data kept separately. Labels name locations without fixing their final byte offsets, which will change when instruction lengths or block positions change. The model returns the complete proposed code listing. The deterministic pipeline then assembles it, restores the preserved data representation, and validates the result. The original binary, this prepared listing, and the final binary are therefore different representations; a similarity score between listings does not measure every change made since the original input.

The experiments request `deepseek-flash`. The locally recorded version name comes from the client's alias table, rather than independent provider confirmation, so the requested identifier is the reproducible model designation available in these records.

9.2 Prompting Conditions and Repeated Attempts

A *zero-shot* request describes the transformation and its constraints without a worked example. A *two-shot* request adds two examples of input and transformed output. The number of shots thus counts examples inside a prompt; it does not count generated variants or execution tests. Each input, transformation family, and shot setting defines an experimental *cell*.

The general pipeline may send a rejected answer back with validation feedback and request a correction. Such a later attempt has information that was absent from the initial request. The benign benchmark disables this mechanism and performs three separate, single-generation invocations per cell, retaining every failure. The full-payload campaign instead targets one output per cell, with some retries and restarted failed runs present in the saved logs. Its results are reported as cell outcomes, not as first-generation success rates. Section 11 gives the exact accounting for both protocols.

9.3 From a Proposed Listing to an Assessed Output

Validation checks whether the response can be assembled and decoded, whether tracked control-flow targets remain consistent, whether preserved data survives, and whether the requested transformation's structural requirements are met. A response can fail at any of these stages, or be truncated before it contains a complete listing. Passing them means that Zone accepts a constructed output; it does not prove behavioral equivalence.

Functional assessment then uses an external *test harness*, meaning the program that launches a test and records its observable result. For the benign fixtures, it compares a complete output buffer with the expected bytes fixed before generation. For the original payloads, it checks the class-specific observable described in Section 11. Neither test selects generations within these two experiments. The earlier MessageBox pilot did use its dialog check during generation and is therefore kept separate. Saved records distinguish model requests, accepted outputs, execution results, time, and estimated cost, allowing rejection and execution failure to be analyzed independently.

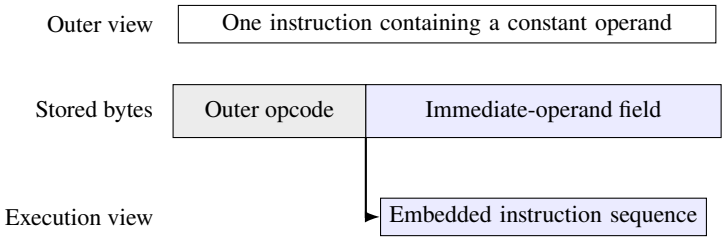


Fig. 9. Conceptual views of a wrapped region. Starting at the outer opcode interprets the operand as a constant; entering the embedded sequence interprets some of those same bytes as instructions. The drawing is not to scale and omits inter-region control transfers.

10 Assembly Wrapping: Mechanism and Integration

10.1 The Same Bytes Can Have Two Instruction Interpretations

Assembly wrapping changes how executable bytes are arranged and interpreted. It uses the variable length of x86 and x64 instructions: decoding the same bytes from different starting positions can produce different instruction sequences. In the existing wrapping technique, bytes belonging to the intended computation are placed within the immediate-operand field of a larger instruction [1]. An *immediate operand* is a constant stored directly in an instruction's encoding. A decoder starting at the outer instruction reads that field as a constant; execution entering the embedded sequence reads it as instructions instead.

Figure 9 illustrates the distinction without relying on a particular encoding. The *container* is the outer instruction whose operand holds the embedded bytes. The execution path connects the embedded sequences in their intended order, whereas a static listing following the container boundaries shows a different instruction sequence. The bytes need not be decrypted to obtain that second interpretation. This is a representation technique based on overlapping instructions, not a new form of encryption or a guarantee that an analyzer cannot recover the executed code.

10.2 What Zone Adds and What the Transformation Requires

The contribution of this work is the integration and assessment of an existing technique, with implementations for both architectures. Zone starts from a prepared instruction stream, groups supported instructions into wrapped regions, preserves the required execution relationships, and updates the supported location-dependent references after layout. Some instructions or local layouts cannot be placed in the main container representation; the implementation retains these in separate, unwrapped regions. An output may therefore contain both wrapped and ordinary code.

The x86 and x64 implementations share this purpose but use different encodings and addressing rules. In particular, the available immediate operand has a different width, affecting which instruction groups fit. The x86 implementation was introduced after the first benign benchmark snapshot and is evaluated under its own recorded version.

Data preparation remains necessary. Wrapping does not determine whether an apparent instruction is actually an embedded string or table. If that earlier interpretation is wrong, construction may be refused or the output may fail when executed. Successful relocation checks likewise cover the references the engine supports, not every possible address computation.

10.3 What the Evaluation Can Establish

Three questions follow from this mechanism: can Zone construct an output for the input, does that output pass its behavioral test, and what do static scanners report for the produced bytes? These questions have different denominators. A scan cannot evaluate an unbuilt output, and an absence of malicious verdicts does not demonstrate that a constructed output works. The following experiments therefore report build availability, functional outcomes, and scan results separately, including the failures on the full 102-payload corpus rather than only the successful benign examples.

11 Experimental Design for the Extensions

11.1 Questions and Test Populations

The extended evaluation asks four questions. Can the identification methods find annotated data without misclassifying code? How often do native and LLM transformations produce outputs that pass a functional test? How does wrapping perform when moving from small controlled programs to the original payload corpus? Finally, what do static scans show once duplicate outputs and unsuccessful executions are accounted for?

We use three populations with different purposes. The *benign functional corpus* contains thirteen fixtures, meaning small programs constructed to exercise specified behavior: seven x64 and six x86 cases cover arithmetic, branches, loops, local calls, mixed control flow, strings, and an additional x64 address-acquisition case. Each fixture writes to a 256-byte output buffer. The expected writes are declared before transformation, and every other byte must retain its initial `0xCC` value. This initial value is a *sentinel*: changing it reveals an unexpected write. A separate C runner returns the complete buffer for comparison. All thirteen unmodified fixtures pass.

The *classification corpus* contains six further fixtures with independent reference data ranges: two programs containing multiple strings (one per architecture), an x64 binary table, two programs containing no data, and an x64 program whose instruction bytes resemble printable text. This last case tests whether a classifier mistakes code for a string. Neither the annotations nor the other classifier's answer is given to the model. These are six distinct inputs; repetitions measure repeated outcomes on those inputs rather than enlarging the program population.

The *original payload corpus* contains the same 38 x64 and 64 x86 input files as the conference study. The current harness observes a dialog for MessageBox, the prescribed connection event for network cases, or clean process exit for the remaining cases. All 102 originals pass these checks. These observables are narrower than complete output-buffer equality: for example, observing a connection does not establish that every later interaction is correct. We use "working" below to mean passing the specified harness, not proven equivalence for every execution.

11.2 Versions, Experimental Cells, and Call Accounting

The code was frozen in three experiment snapshots. B1 (13 November 2026, 13:07 CET) supplies the benign native matrix and x64 LLM tests. B2 (14:05 CET) supplies the x86 LLM tests, x86 wrapping port, and classifier tests. B3 supplies the full-payload campaign; the hashed rewriting and LLM modules match B2. Consequently, a comparison between B1 x64 and B2 x86 does not isolate architecture alone.

The benign native matrix uses three runs per input and family. The benign LLM matrix uses four families, two prompting conditions, and three single-generation invocations per cell: $7 \times 4 \times 2 \times 3 = 168$ x64 invocations and $6 \times 4 \times 2 \times 3 = 144$ x86 invocations. Rejected responses and transport failures remain in these totals; none is replaced. Classification uses three LLM invocations and one symbolic run per annotated fixture.

On the original corpus, wrapping uses three recorded randomization seeds per input, giving $102 \times 3 = 306$ planned variants. It is assessed alone after data preparation. The LLM matrix has $102 \times 4 \times 2 = 816$ cells, targeting one accepted output per cell. The output-token ceiling is 65,536. The saved ledger contains 830 run records: fourteen failed cells were restarted and failed again. Four run records contain multiple generations; the other records with known counts contain one. In total, 835 API calls are recorded, with usage missing for one further run record. Two accepted outputs required a second generation. Thus this completed campaign is not a uniform one-generation-per-cell experiment. Cell-level reporting retains all 816 cells, while cost accounting includes all 830 run records.

The data-identification payload comparison selects nineteen inputs (thirteen x64, six x86) for which symbolic preparation extracts movable data. Both methods use the shared deterministic handling and the same functional harness. This selection evaluates their behavior on those inputs; it cannot estimate identification accuracy across all 102 inputs.

11.3 Construction, Functionality, and Identification Measures

Let N be the number of planned cases, B the number of constructed outputs, and W the number passing the functional test. We distinguish

$$\begin{aligned} \text{build rate} &= B/N, \\ \text{overall working rate} &= W/N, \\ \text{working rate among builds} &= W/B. \end{aligned}$$

For example, if ten cases yield eight binaries and six pass, these rates are 80%, 60%, and 75%, respectively. Reporting only the last value would omit the two cases for which no binary was produced. Retesting a failed binary does not create a new variant. U denotes the number of distinct output SHA-256 hashes, a measure of byte-level uniqueness.

For annotated fixtures, exact identification means that all reported data intervals equal the reference intervals, with no missing or extra region. We also count missed data bytes and code bytes incorrectly labeled as data. Writing P for predicted data bytes and G for reference data bytes, data precision is $|P \cap G|/|P|$ and recall is $|P \cap G|/|G|$. Inputs without reference data are evaluated as empty-set decisions; undefined ratios are

not replaced by favorable scores. On unannotated payloads we report interval agreement and final-output identity, not accuracy. The available comparison matches symbolic extracted ranges against reported LLM proposals, so it also compares different processing stages.

11.4 Representation, Static Scans, and Resource Use

LLM sequence similarity compares the prepared input listing with an accepted output using normalized mnemonic-and-operand signatures. The `SequenceMatcher` ratio is $S = 2M/(u+v)$, where M is the number of matched elements and u, v are the sequence lengths [21]. A larger value means more sequence overlap, not greater behavioral correctness. NOP-like instructions are removed in the NOP condition; therefore $S = 1$ can coexist with changed binary bytes. Means include accepted outputs only. This measure differs from the historical cosine and entropy measurements in Section 5.

Static scanning has two units of analysis. For a file with m_i malicious engine responses and a_i total reported responses, $d_i = m_i/a_i$. Across files, q is the fraction with at least one malicious response. For example, one malicious response among 75 gives $d_i = 1/75$, but the file counts as detected when computing q. The present nonempty reports contain 75 responses each, including unsupported formats, timeouts, and failures; those categories are disclosed rather than interpreted as benign verdicts. An empty report is not an evaluated negative and has no d_i.

Scans are deduplicated by hash. A hash shared by multiple transformation conditions retains membership in each condition, so adding table rows does not give the size of their union. We also separate passing and failing LLM outputs before interpreting their scan results. Historical, earlier pilot, and current scans remain separate cohorts because their inputs, selection, and collection periods differ.

Time and token usage come from saved records; costs are local tariff estimates, not invoices. We include failed and restarted runs in campaign costs and do not treat run duration as pure API latency. The accompanying evidence freezes the completed logs available on 14 November 2026, including scan records through 01:17 CET, with source hashes and derived tables. Earlier benchmark evidence is retained under its original snapshot.

12 Results of the Extended Evaluation

The new results show a consistent distinction between small controlled tests and the original payload corpus. The benign programs usually survive transformation, whereas construction failures and execution failures become substantial in the broader corpus. We first examine identification and controlled behavior, then the completed wrapping and LLM campaigns, and finally the scans of the outputs those campaigns produced.

12.1 Can the Methods Identify Annotated Data?

On the six classification fixtures, all eighteen LLM invocations and all six symbolic runs return the exact reference interval sets (Table 7). Both methods miss zero data bytes and

Table 7. Independent classifier fixtures. *D* is annotated data bytes. Exact interval sets are compared against predeclared references. Mean times are per run in seconds.

Arch.	Case	*D*	LLM exact	Symbolic exact	LLM time	Sym. time
x64	databait	0	3/3	1/1	4.6	2.4
x64	multidata	11	3/3	1/1	4.2	2.6
x64	nodata	0	3/3	1/1	2.1	2.4
x64	table	16	3/3	1/1	2.6	2.4
x86	multidata	11	3/3	1/1	4.4	2.5
x86	nodata	0	3/3	1/1	1.9	2.3

classify zero code bytes as data. The three fixtures containing data have precision and recall equal to one; the other three correctly return no data. All resulting executions pass their buffer checks. This includes the deliberately misleading printable-code case, which is rejected as data rather than extracted as a string.

These results establish correctness on the six annotated examples, not on arbitrary payloads. The nineteen selected original payloads illustrate why that distinction matters. Both pipelines build 19/19 outputs and pass 17/19 harness checks. Their compared intervals agree in only 9/19 cases, while their final binaries are byte-identical in 16/19. Shared filtering and reference handling can therefore remove differences between proposals. Without independent annotations, neither 9/19 nor 16/19 measures classifier accuracy. The same two inputs fail in both arms, which is consistent with a shared downstream limitation but does not by itself isolate its cause.

The recorded median durations are 123.95 seconds for symbolic preparation and 59.88 seconds for the LLM arm. The latter's estimated cost is $0.2218 over nineteen runs. Different preparation and caching work prevents a controlled general speedup claim. On the separate six-fixture benchmark, the total LLM cost is approximately $0.0059. An earlier three-case benign check records another 9/9 exact LLM results; those repetitions are not added to the six-fixture population.

12.2 Do Transformations Preserve the Benign Fixtures' Outputs?

Table 8 reports the native engine tests. For x64, each family passes 21/21 runs (seven fixtures, three runs each). For x86, EQ, NOP, and BCF pass 18/18. B1 IBP builds 17/18 binaries but none passes the complete-buffer check; the remaining case raises a construction exception. The B2 x86 wrapping implementation passes all 18/18. A constructed binary can thus fail even these controlled observables, as the IBP row demonstrates.

The LLM benchmark yields accepted outputs with correct buffers in 167/168 x64 invocations and 138/144 x86 invocations. Every accepted output passes the external test. The seven unsuccessful invocations comprise five family-specific validation rejections and two transport errors, rather than seven executions with incorrect buffers. Table 9 keeps these failures in each condition's denominator.

Across conditions, x64 produces 108 distinct hashes at an estimated cost of $0.5205; x86 produces 99 at $0.5132. Hashes shared across conditions explain why the union is smaller than the sum of row-wise uniqueness. EQ frequently repeats outputs. NOP's

Table 8. New native fixture results. *N*: attempts; *B*: builds; *W*: correct complete buffers; *U*: distinct built-output hashes. B1/B2 identify different frozen engine versions.

Arch.	Family	Version	*N*	*B*	*W*	*U*
x64	BCF	B1	21	21	21	21
x64	EQ	B1	21	21	21	16
x64	IBP	B1	21	21	21	17
x64	NOP	B1	21	21	21	21
x64	WRAP	B1	21	21	21	21
x86	BCF	B1	18	18	18	18
x86	EQ	B1	18	18	18	14
x86	IBP	B1	18	17	0	16
x86	NOP	B1	18	18	18	18
x86	WRAP	B2	18	18	18	18

similarity of 1.000 follows from filtering NOP-like operations before matching; it does not indicate that the binaries are identical.

For x86 IBP, the two-shot condition passes 17/18 invocations compared with 13/18 for zero-shot. For x64 IBP, the corresponding outcomes are 20/21 and 21/21. These are observations on six or seven fixtures under the stated versions; they do not demonstrate a general advantage for two examples.

12.3 Wrapping on All 102 Original Payloads

The full B3 campaign attempts 306 variants, three per input. It constructs 243 binaries (79.4% of planned cases), of which 170 pass the harness. Thus $W/N = 55.6\%$, whereas $W/B = 70.0\%$. The distinction reveals a substantial coverage gap that would be hidden by reporting only the successful benign tests.

The two architectures fail at different stages. On x64, 51 attempts do not build, while all 63 constructed outputs pass. On x86, only twelve attempts do not build, but 73 of the 180 constructed outputs fail the harness. All 73 also fail the saved isolated retests; those retests are not counted as additional variants. There are 207 distinct hashes among the 243 binaries, showing that different seeds do not always produce different bytes.

The logs include incomplete instruction interpretation and reference consistency errors during construction. These symptoms accord with the method's dependence on data preparation and supported relocation. They do not establish a single cause for every execution failure. The earlier selected x64 experiment, which built 200/260 variants and passed 198/200 constructed outputs, concerns a different and smaller population and is not substituted for the full-corpus results.

12.4 Completed LLM Transformation on the Original Corpus

All 816 planned cells have an outcome. The campaign constructs 350 outputs, and 275 pass the external harness: a build rate of 42.9%, an overall working rate of 33.7%, and

Table 9. Benign DeepSeek transformation. W/N includes rejected and failed invocations. S is mean sequence similarity over accepted outputs; U/B is unique hashes per built output within the condition. Time is mean invocation seconds; cost is the sum of local USD estimates.

Arch.	Family	Shots	W/N	S	U/B	Time	Cost
x64	BCF	0	21/21	0.574	18/21	43.0	0.107
x64	BCF	2	21/21	0.575	20/21	35.6	0.088
x64	EQ	0	21/21	0.831	12/21	28.8	0.064
x64	EQ	2	21/21	0.832	11/21	29.4	0.068
x64	IBP	0	21/21	0.783	14/21	25.5	0.062
x64	IBP	2	20/21	0.779	15/20	30.8	0.077
x64	NOP	0	21/21	1.000	18/21	11.9	0.027
x64	NOP	2	21/21	1.000	20/21	11.7	0.028
x86	BCF	0	18/18	0.573	18/18	47.9	0.103
x86	BCF	2	18/18	0.580	17/18	43.8	0.095
x86	EQ	0	18/18	0.852	11/18	29.0	0.056
x86	EQ	2	18/18	0.843	11/18	33.3	0.067
x86	IBP	0	13/18	0.705	9/13	31.0	0.052
x86	IBP	2	17/18	0.739	12/17	32.5	0.071
x86	NOP	0	18/18	1.000	18/18	15.1	0.032
x86	NOP	2	18/18	1.000	18/18	18.6	0.039

Table 10. Full original-corpus wrapping campaign, B3. The harness observables are distinct from the benign exact-buffer oracle. U counts distinct constructed hashes.

Arch.	Inputs	B/N	W/N	W/B	U
x64	38	63/114	63/114	63/63	63
x86	64	180/192	107/192	107/180	144

a working rate among builds of 78.6%. Table 11 shows the complete architecture/family/shot breakdown. These are pipeline outcomes, including the retries described in Section 11.

On x64, only 53/304 cells produce binaries, but 52/53 of those binaries pass. On x86, 297/512 produce binaries and 223/297 pass. Construction is therefore the principal observed bottleneck on x64, while x86 also has a substantial failure rate after construction. The aggregate success on benign fixtures does not predict these corpus-level outcomes.

Zero-shot produces 178 builds and 139 passing outputs from 408 cells; two-shot produces 172 builds and 136 passing outputs from 408 cells. Their overall working rates are 34.1% and 33.3%. These descriptive differences offer no consistent evidence that adding two examples improves the outcome. They also concern repeated configurations of the same inputs, not 816 independent programs.

The 466 failed cells have recorded error categories comprising 245 disassembly/boundary errors, 74 validation errors, 53 output-length terminations, one transport error, and 93 other errors. These labels describe the observed failure stage, not proven root causes. In particular, truncation means the response reaches its output limit before

Table 11. Completed B3 LLM payload matrix. *N*: planned cells; *B*: accepted builds; *W*: builds passing the external harness; *U*: distinct built-output hashes within the condition. Every planned cell is retained.

Arch.	Family	Shots	*N*	*B*	*W*	*U*
x64	EQ	0	38	7	7	7
x64	EQ	2	38	6	6	6
x64	NOP	0	38	8	8	8
x64	NOP	2	38	6	6	6
x64	BCF	0	38	6	6	6
x64	BCF	2	38	6	5	6
x64	IBP	0	38	6	6	6
x64	IBP	2	38	8	8	8
x86	EQ	0	64	34	25	30
x86	EQ	2	64	36	28	34
x86	NOP	0	64	38	28	38
x86	NOP	2	64	35	27	35
x86	BCF	0	64	40	31	40
x86	BCF	2	64	36	26	36
x86	IBP	0	64	39	28	36
x86	IBP	2	64	39	30	38

delivering a complete acceptable listing; it is distinct from a generated program failing at execution time.

The 350 built outputs contain 323 distinct hashes. Their mean recorded sequence similarity is 0.906. The median retained-run duration is 160.3 seconds, including local processing. Summing the representative record for each cell gives an estimated $19.2700, but including the fourteen restarted failed runs raises recorded campaign cost to $19.6258. The latter is the appropriate total for work actually recorded; missing usage for one run prevents claiming complete billing coverage.

12.5 Static Scans of the Completed Campaign

The current manifest contains 612 file entries: 243 wrapping outputs, 350 LLM-transformed outputs, and nineteen outputs from LLM data handling. They correspond to 549 distinct hashes. Every hash has a record marked completed, but one LLM BCF zero-shot record contains no engine responses. We exclude that empty result from measured scan rates: 548 hashes are evaluable, with 75 reported responses each. Table 12 retains each hash in every condition to which it belongs.

None of the 207 wrapping hashes receives a malicious response. Across their 15,525 reported responses, however, 3,090 are format-unsupported, 132 time out, and 29 fail. The observation is therefore an absence of malicious verdicts for those submitted raw files, not 75 successful benign classifications per file. It also applies to constructed outputs, including ones that fail the functional harness; it is not a result for all 306 planned variants.

Table 12. Current raw-file scans. U: distinct submitted hashes; V: hashes with nonempty results; D: hashes with a malicious response; $q = D/V$; $\overline{m}$: mean malicious-response count over V. The empty BCF/0 report is excluded from V.

Condition	U	V	D	q (%)	$\overline{m}$
Wrapping	207	207	0	0.0	0.00
LLM EQ / 0	37	37	26	70.3	2.49
LLM EQ / 2	40	40	30	75.0	2.80
LLM NOP / 0	46	46	18	39.1	2.17
LLM NOP / 2	41	41	15	36.6	1.78
LLM BCF / 0	46	45	36	80.0	3.84
LLM BCF / 2	42	42	29	69.0	5.24
LLM IBP / 0	42	42	37	88.1	5.48
LLM IBP / 2	46	46	40	87.0	5.80
LLM data only	19	19	19	100.0	20.63

For LLM transformation, 221 of 322 evaluable hashes receive a malicious response ($q = 68.6\%$), with mean 3.80 malicious responses per hash. The additional submitted hash has an empty report. The 275 passing files correspond to 263 hashes, of which 210 are detected (79.8%, mean 4.41). The 75 failing files correspond to sixty hashes, of which 59 have usable reports and eleven are detected (18.6%, mean 1.05). The lower scan rate of failing outputs must not be interpreted as successful behavioral preservation with reduced detection. It instead shows why functionality and verdicts must be linked when interpreting the experiment.

All nineteen LLM data-handling hashes receive malicious responses, with mean 20.63. That arm preserves supported data and references without requesting one of the four instruction-transformation families. It answers a different experimental question from LLM morphing. Across the whole current scan cohort, 8,184 responses are format-unsupported, 298 time out, and 74 fail; all remain visible in the response denominator.

12.6 Relationship to Earlier Pilots and Scan Results

The earlier 2026 cohort has 239 file entries and 220 distinct hashes, all with nonempty completed reports. Twelve hashes receive at least one malicious response. Table 13 retains its original group membership and is separate from the current 549-hash manifest.

In that selected x64 cohort, none of the 197 distinct wrapping outputs receives a malicious verdict. Thirteen contemporaneous original controls all do, with mean 26.62/75 responses. Across the earlier cohort's 16,500 mutant-engine responses, 3,259 are format-unsupported, 283 time out, and 73 fail. The thirteen controls do not constitute a matched scan baseline for every input and configuration in the new full-corpus campaign.

The earlier MessageBox transformation pilot retains 24 passing final outputs, but its dialog check already participated in generation-time selection and a failed run was restarted. Its 24/24 therefore measures selected artifacts, not the single-invocation protocol of the benign benchmark. No benign fixture from that benchmark was submitted for scanning. The historical archive also has missing local artifacts for 6,972 of 15,021

Table 13. Completed earlier 2026 raw-file cohort. q is detected hashes over completed hashes; $\overline{d}$ is the percentage of all reported engine responses that are malicious, averaged per file. $\overline{m}$ is mean malicious-engine count. These are separate measures.

Group	Hashes	q	$\overline{d}$ (%)	$\overline{m}$
LLM data only	4	4/4	23.33	17.50
LLM data + EQ	1	0/1	0.00	0.00
LLM BCF / 0	3	1/3	1.33	1.00
LLM BCF / 2	3	3/3	6.67	5.00
LLM EQ / 0	2	0/2	0.00	0.00
LLM EQ / 2	2	0/2	0.00	0.00
LLM IBP / 0	2	1/2	3.33	2.50
LLM IBP / 2	3	3/3	3.11	2.33
LLM NOP / 0	3	0/3	0.00	0.00
LLM NOP / 2	3	1/3	2.22	1.67
Wrapping (prior x64)	97	0/97	0.00	0.00
Wrapping combined (prior x64)	100	0/100	0.00	0.00

mutant records, and its 102 original records represent ninety distinct hashes. These facts limit retrospective verification and independence without turning the published results into newly repeated experiments.

13 Discussion of the Extensions

The experiments answer different parts of the rewriting problem. Exact classification on the six annotated fixtures shows that the two identification methods handle those controlled layouts, including a printable-code counterexample. The selected payload comparison shows why classification and data handling must still be assessed separately: different interval proposals often converge to the same output after deterministic processing. More broadly representative annotations would be needed to quantify identification accuracy beyond these examples.

The strongest functional results occur on the benign corpus, whose observable behavior is specified precisely. Both wrapping and LLM transformation then lose substantial coverage on the original payloads. On x64 the principal measured obstacle is obtaining a constructed output; on x86 some accepted outputs also fail execution checks. These outcomes support retaining all three stages—interpretation, construction, and execution—in the evaluation. A change that increases build availability cannot be assumed to improve the other stages.

Wrapping produces a marked difference in static representation, and its 207 submitted hashes receive no malicious responses in this scan cohort. The experiment does not isolate that representation as the cause of every verdict: format support, failed analyses, and input selection also affect the observations. Likewise, the apparent scan advantage of unsuccessful LLM outputs disappears as evidence of useful transformation once their failed functional checks are considered. These measurements concern raw files and pre-

scribed harness observables, not a deployed endpoint protection system's response to a complete application.

The study is descriptive. The same inputs occur in multiple conditions; some inputs and outputs share hashes; the benign architectures use different frozen versions; and the full LLM campaign includes nonuniform retry histories. These dependencies prevent treating every cell as an independent sample or attributing small differences solely to prompting. The source snapshots, per-cell outcomes, hashes, and separate cost totals make those limitations inspectable. The completed results identify the remaining reliability gap without presenting structural validation or a successful limited observable as a proof of semantic equivalence.

14 Related Work

Instruction-level obfuscation has been studied in both offensive and defensive settings [26,6]. Earlier engines, including W95/Regswap [8] and MetaPHOR [24], explored register substitution and code reorganization. Instruction substitution, insertion, and control-flow transformations underpin the techniques summarized here [7,8,23,10,15].

Polymorphic encoding provides a related but distinct representation strategy. Analyses of SGN describe both the variation in its decoding stub and opportunities to recognize it [9,18]; memory-extraction research addresses the body recovered during execution [11]. For metamorphic programs, detection studies have considered Hidden Markov Models [27,8], API-trace similarity [29], and genetic-algorithm-based methods [14].

More recent work examines LLM-based assembly transformation, including the MetamorphASM benchmark and its MAD dataset [19,16]. Compiler-based systems such as Obfuscator-LLVM [15] offer another setting, with access to source-level or intermediate information during compilation.

The contribution of the conference paper was its payload-level *bin2bin* study, the PII design, and the joint evaluation of selected transformation configurations [4]. Those contributions are the starting point of this manuscript, rather than new claims of the extended version.

The extensions additionally connect data identification with bounded symbolic and static analysis [2], and distinguish LLM classification from assembly transformation. Assembly wrapping is attributed to its existing description and acknowledged antecedents in overlapping instructions [1]; its integration into Zone is assessed as an implementation contribution. These new directions build on the earlier payload-level study while preserving its detailed results for comparison.

15 Conclusions

The extended Zone pipeline makes data identification and supported reference handling explicit before instruction transformation. It offers two distinct LLM roles—proposing data regions and proposing rewritten assembly—and integrates assembly wrapping for both x64 and x86. The original PII design and conference experiments remain the foundation of this broader assessment.

The controlled tests give exact identification on six annotated fixtures and correct buffers in 167/168 x64 and 138/144 x86 LLM invocations. The completed original-corpus campaigns show the limits of extending those results: wrapping passes 170/306 planned cases, while LLM transformation passes 275/816 cells. Construction and execution failures contribute differently across architectures. Two-shot prompting does not show a consistent advantage over zero-shot in the observed outcomes.

None of the 207 submitted wrapping hashes receives a malicious verdict in the current raw-file scans. For LLM transformation, 221/322 evaluable hashes do, with one further empty scan report. Relating these measurements to functionality is essential: LLM outputs that fail the harness have substantially lower static detection than those that pass. The resulting evidence supports separate assessment of classification, construction, behavior, and representation; a favorable result at one stage cannot stand in for success at the others.

Acknowledgments. This work was partially funded by the University of Perugia's project "Fighting Cybercrime with OSINT" (FICO), Civil Safety and Security for Society (WP2).

Disclosure of Interests. *To be completed by the authors before submission.*

References

1. 0x4ndr3: Assembly wrapping: A technique for anti-disassembly. Author's technical article, June 8 (2020), https://0x4ndr3.github.io/posts/assembly-wrapping-a-new-technique-for-anti-disassembly/, accessed 13 November 2026
2. angr Project: Control-flow graph recovery. Official angr documentation, https://docs.angr.io/en/latest/analyses/cfg.html, accessed 13 November 2026
3. Austin, T.H., Filiol, E., Josse, S., Stamp, M.: Exploring hidden markov models for virus analysis: A semantic approach. In: 2013 46th Hawaii International Conference on System Sciences. pp. 5039–5048 (2013). https://doi.org/10.1109/HICSS.2013.217
4. Bistarelli, S., Di Benedetto, J., Santini, F.: On automating the obfuscation of malware payloads: Zone, a bin2bin metamorphic engine. In: Proceedings of the 23rd International Conference on Security and Cryptography. vol. 1, pp. 216–227. SCITEPRESS (2026). https://doi.org/10.5220/0015170600004103
5. Campion, M., Dalla Preda, M., Giacobazzi, R.: Learning metamorphic malware signatures from samples. J. Comput. Virol. Hacking Tech. **17**(3), 167–183 (2021)
6. Cho, I., Towey, D., Kar, P.: Using obfuscators to test compilers: A metamorphic experience. In: 47th IEEE Annual Computers, Software, and Applications Conference, COMPSAC 2023. pp. 1786–1791. IEEE (2023)
7. Desai, P.: Towards an Undetectable Computer Virus. Master's project, San Jose State University (2008). https://doi.org/10.31979/etd.z942-squp, https://scholarworks.sjsu.edu/etd_projects/90/
8. Desai, P., Stamp, M.: A highly metamorphic virus generator. International Journal of Multimedia Intelligence and Security **1**(4), 402–427 (2010)
9. EgeBalci: Shikata Ga Nai encoder ported into Go with several improvements (2023), https://github.com/egebalci/sgn
10. es3n1n: Obfuscating native code for fun: Part 1 - introduction (2023), https://blog.es3n1n.eu/posts/obfuscator-pt-1/

11. Farley, R., Wang, X.: CodeXt: automatic extraction of obfuscated attack code from memory dump. In: Information Security: 17th International Conference, ISC 2014, Hong Kong, China, October 12-14, 2014. Proceedings 17. pp. 502–514. Springer (2014)
12. Geng, J., Wang, J., Fang, Z., Zhou, Y., Wu, D., Ge, W.: A survey of strategy-driven evasion methods for PE malware: Transformation, concealment, and attack. Comput. Secur. **137**, 103595 (2024)
13. Glynos, D.A.: Context-keyed payload encoding: Fighting the next generation of IDS. In: Athens IT Security Conference (ATH. C0N) (2010)
14. Javaheri, D., Lalbakhsh, P., Hosseinzadeh, M.: A novel method for detecting future generations of targeted and metamorphic malware based on genetic algorithm. IEEE Access **9**, 69951–69970 (2021)
15. Junod, P., Rinaldini, J., Wehrli, J., Michielin, J.: Obfuscator-LLVM: Software protection for the masses. In: 2015 IEEE/ACM 1st International Workshop on Software Protection. pp. 3–9. IEEE (2015)
16. Madani, P.: Metamorphic malware evolution: The potential and peril of large language models. In: 5th IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications, TPS-ISA 2023, Atlanta, GA, USA, November 1-4, 2023. pp. 74–81. IEEE (2023)
17. Madenur Sridhara, S., Stamp, M.: Metamorphic worm that carries its own morphing engine. J. Comput. Virol. **9**(2), 49–58 (2013)
18. Miller, S., Reese, E., Carr, N.: Shikata Ga Nai encoder still going strong (2019), https://cloud.google.com/blog/topics/threat-intelligence/shikata-ga-nai-encoder-still-going-strong/
19. Mohseni, S., Mohammadi, S., Tilwani, D., Saxena, Y., Ndawula, G.K., Vema, S., Raff, E., Gaur, M.: Can LLMs obfuscate code? a systematic analysis of large language models into assembly code obfuscation. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 39, pp. 24893–24901. AAAI Press (2025). https://doi.org/10.1609/aaai.v39i23.34672
20. Nawaz, M.S., Fournier-Viger, P., Nawaz, M.Z., Chen, G., Wu, Y.: MalSPM: Metamorphic malware behavior analysis and classification using sequential pattern mining. Comput. Secur. **118**, 102741 (2022)
21. Python Software Foundation: difflib: Helpers for computing deltas. Python documentation, https://docs.python.org/3/library/difflib.html, accessed 13 November 2026
22. Sathyanarayan, V.S., Kohli, P., Bruhadeshwar, B.: Signature generation and detection of malware families. In: Information Security and Privacy: 13th Australasian Conference, ACISP 2008, Wollongong, Australia, July 7-9, 2008. Proceedings 13. pp. 336–349. Springer (2008)
23. Szor, P., Ferrie, P.: Hunting for metamorphic. In: Virus bulletin conference (2001)
24. The Mental Driller: Metamorphic permutating high-obfuscating reassembler (2002), http://dsr.segfault.es/stuff/website-mirrors/29A/29a-6/29a-6.602
25. Venkatesan, A.: Code Obfuscation and Virus Detection. Master's project, San Jose State University (2008), https://scholarworks.sjsu.edu/etd_projects/116/
26. Vishwas, G., Nithin, N.S., Varshith, P., Belwal, M.: Unveiling the world of code obfuscation: A comprehensive survey. In: 2023 7th International Conference on Computation System and Information Technology for Sustainable Solutions (CSITSS). pp. 1–8. IEEE (2023)
27. Wong, W., Stamp, M.: Hunting for metamorphic engines. Journal in Computer Virology **2**, 211–229 (2006)
28. Zeng, D., Tan, G.: From debugging-information based binary-level type inference to CFG generation. In: Proceedings of the Eighth ACM Conference on Data and Application Security and Privacy. pp. 366–376. CODASPY '18, Association for Computing Machinery, New York, NY, USA (2018). https://doi.org/10.1145/3176258.3176309
29. Zhang, Q., Reeves, D.S.: MetaAware: Identifying metamorphic malware. In: Twenty-Third Annual Computer Security Applications Conference (ACSAC 2007). pp. 411–420. IEEE (2007)